```
  val source: Source[String, NotUsed] = readJournal.persistenceIds    ❷

  source.runForeach(println)                                          ❸
}
```

❶ Obtains a JDBC Implementation of ReadJournal
❷ Creates a Producer
❸ Executes the stream

As useful as `.persistenceIds` may be for understanding how the API works, it is not the most commonly used method. That's `.eventByTag`. It allows you to filter events, which is usually done for two reasons. The most obvious is that you're only interested in a certain type of event, such as `CargoAdded`. This is the only event currently implemented, but if you extended the example, you would probably add `CargoRemoved` or `ContainerShipped`. The less obvious reason to filter events is to tag events for scaling out. So far there are only two tags, `container-tag-0` and `container-tag-2`, as shown in table 10.3.

Table 10.3 Tags from table `event_tag`

event_id	tag
1	container-tag-0
2	container-tag-0
3	container-tag-2

`container-tag-0` points to the events from the journal with `ordering` values `1` and `2` that have the `persistence_id` value `spcontainer-type-key|9`, while tag `container-tag-2` points to ordering `3` and `persistence_id` value `spcontainer-type-key|11`. Thanks to these tags, when using .`eventsByTag`, you can create two different streams to read the two tags in parallel, maybe on different machines.

To read events by tag, specify the tag you are looking for and the offset. As you know, using `Offset.noOffset` sets the offset to zero, which reads from the beginning of the journal. The output is a `Source[EventEnvelope]`:

```
class PersistenceQuerySpec ... {
  ...
  "a persistence query" should {
    "retrieve the events from db and printing them" in {
      ...
      val source: Source[EventEnvelope, NotUsed] =
        readJournal.eventsByTag("container-tag-0", Offset.noOffset)   ❶
      ...
   }}
  ...
}
```

❶ Passes the tag and offset

This query returns the following `EventEnvelope` from the journal:

```
EventEnvelope(
   akka.persistence.query.Sequence(1),                                ❶
```